

sysdig

Winning the AI Race: **Verifiability is All You Need**

Sergej Epp

Chief Information Security Officer



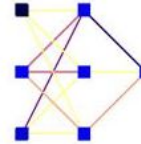
My first experience with Neural Networks

JoeBot for CounterStrike (2001)



Eingabe

Entfernung-a° links
Entfernung Mitte
Entfernung a° rechts



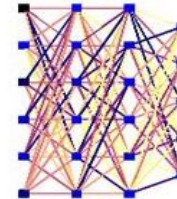
Ausgabe

Bewegung :

- links (<-.5)
- rechts (>.5)

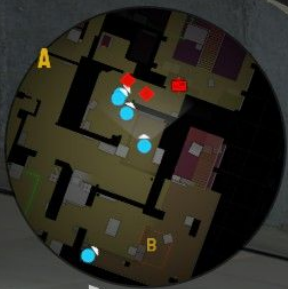
Eingaben

„Gesundheit“ des Bots
Entfernung des Gegners
Beschr. der Waffe des G.
Beschr. der akt. Waffe
Menge der akt. Munition
Momentane Situation



Ausgaben

Springen
Ducken
Verstecken
Links / Rechts
Laufen / Gehen



\$10000

BOT Enemies 100%
BOT War 100%
BOT Niles 100%



sysdig 3



Why AI Security Failed Before GPT

What worked

- Malware / phishing detection → millions of public, labeled samples
- Stable data distributions (malware looks the same everywhere)

What failed

- Insider threat detection (private, sparse, no labels)
- Stealthy, multi-step “living off the land” attacks
- Novel malware families, bespoke social engineering
- Anything needing deep semantic reasoning

Key Challenge: ML couldn't generalize from **public training data** to **private enterprise context**.
Without crisp oracles, defense stayed noisy and brittle.

Supervised vs Reinforcement Learning: Why It Changes the Game in Cyber



Supervised ML

- Trains on labeled data: needs millions of examples
- Hard in cyber: rare attacks, noisy SOC labels, private data
- *Analogy: Homework with no answer key*
- Outcome: brittle, biased, outdated models

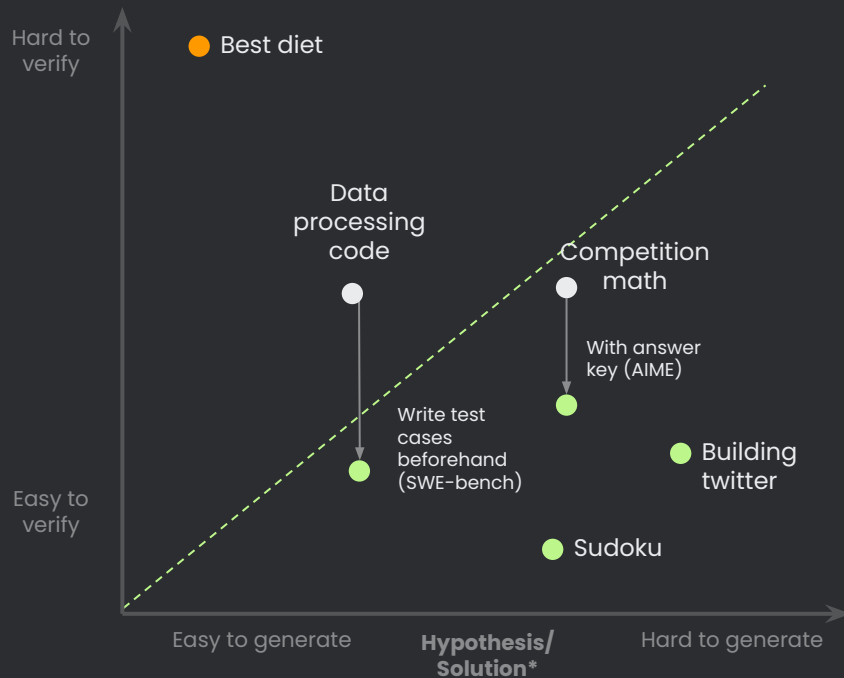


Reinforcement Learning

- Trains on rewards: only needs a verifier (oracle/scoreboard)
- Natural in cyber offense: exploit works or doesn't, data exfiltrated or not
- *Analogy: Video game with a scoreboard*
- Outcome: scalable learning, if oracles are crisp

The Asymmetry of Verification

AI will win where verification is easy



Defining factors

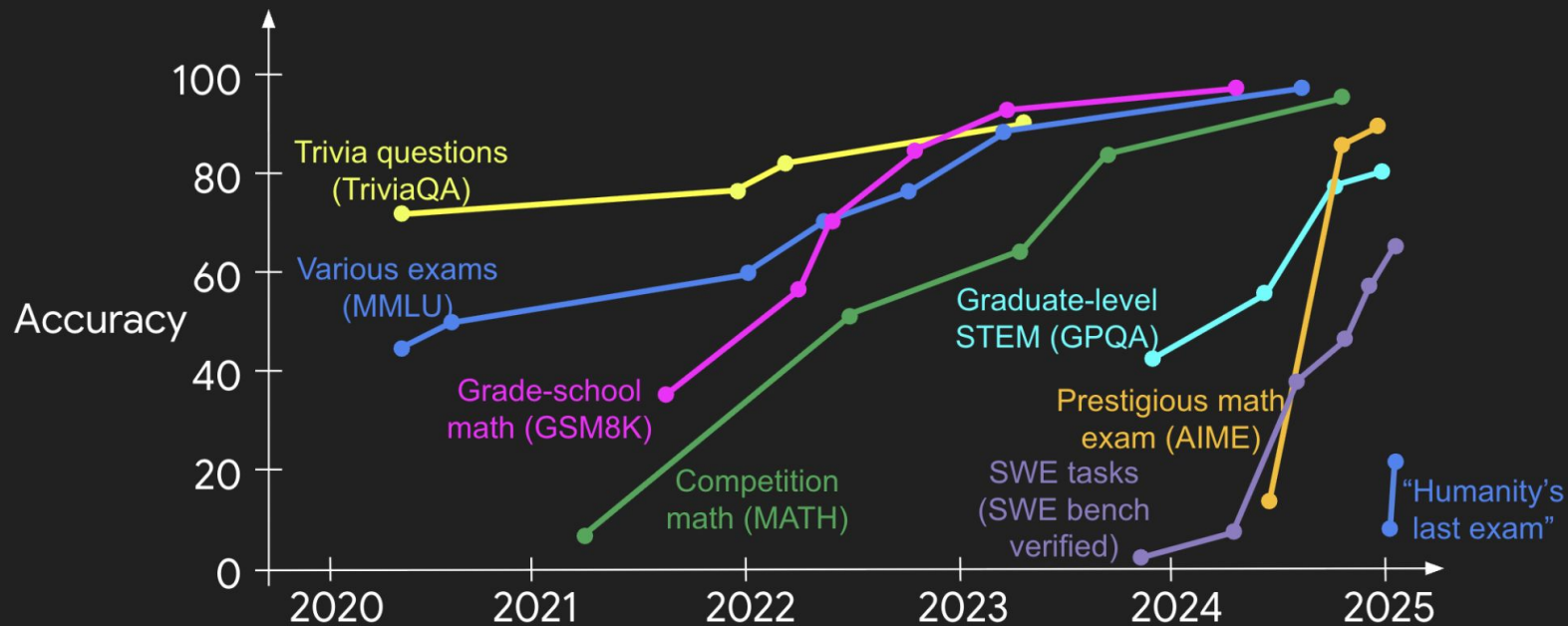
1. LLMs generate (knowledge) for us, but the verification process cannot be offloaded, only pushed up to a different layer
2. Good mechanical verifiers are deterministic
3. Static vs. Dynamic Verification
4. Cost of Verification
 - Available tools
 - Amount of data
 - Expertise of the verification
 - Tolerance of error

Verifier's law:

*The ease of training AI to solve a task is **proportional** to how verifiable the task is.*

*All tasks that are **possible to solve and easy to verify** will be solved by AI.*

Progress on AI benchmark in the past five years



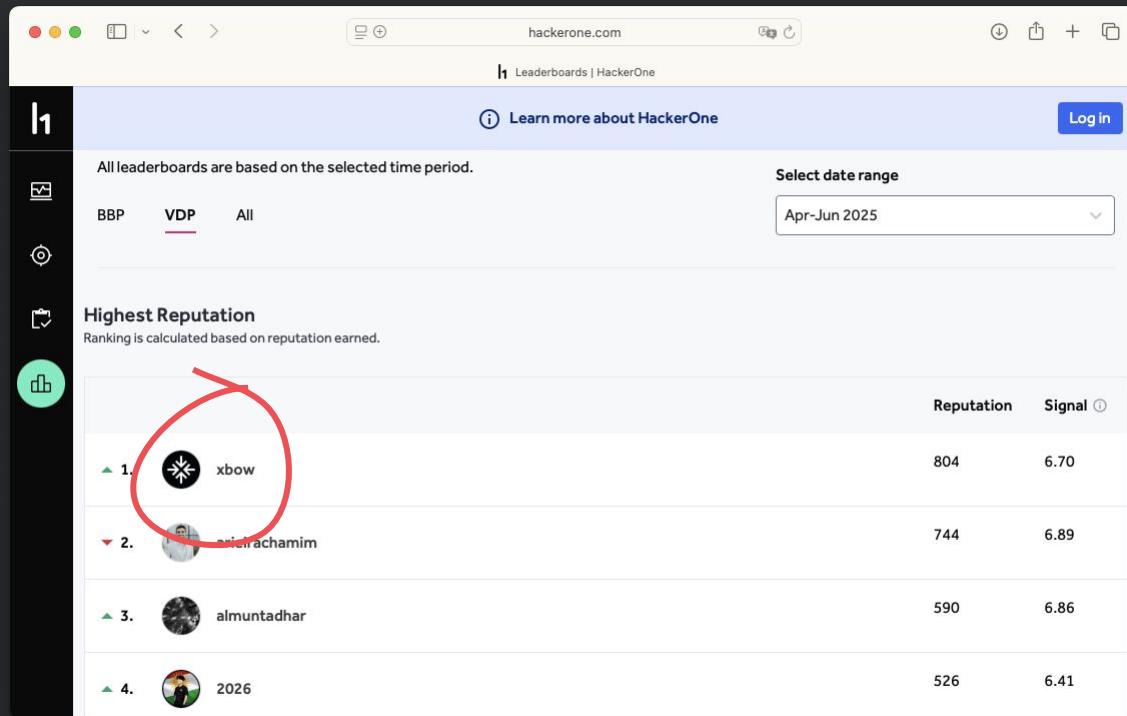
CTFs: AI Cracks What Humans Took Months

In 8 Months from Solving 20% (GPT4) to over 90% (o3)



*OpenAI 03,04 SystemCard: Evaluation against common CTF's Web Application Exploitation, Reverse Engineering, Binary and Network Exploitation, Cryptography, Miscellaneous.

HackBots Are Taking Over Tiny Finds, Tremendous Value



The screenshot shows the HackerOne website's 'Leaderboards' section. The page title is 'Leaderboards | HackerOne'. A navigation bar includes a 'Log in' button and a link to 'Learn more about HackerOne'. Below the navigation bar, there's a section for 'All leaderboards are based on the selected time period.' with a 'Select date range' dropdown set to 'Apr-Jun 2025'. The main section is titled 'Highest Reputation' and includes a note: 'Ranking is calculated based on reputation earned.' A table displays the top four users by reputation. The first two users, 'xbow' and 'achamim', are circled in red. The table columns are 'Rank', 'User', 'Reputation', and 'Signal'.

		Reputation	Signal ⓘ
1.	xbow	804	6.70
2.	achamim	744	6.89
3.	almuntadhar	590	6.86
4.	2026	526	6.41

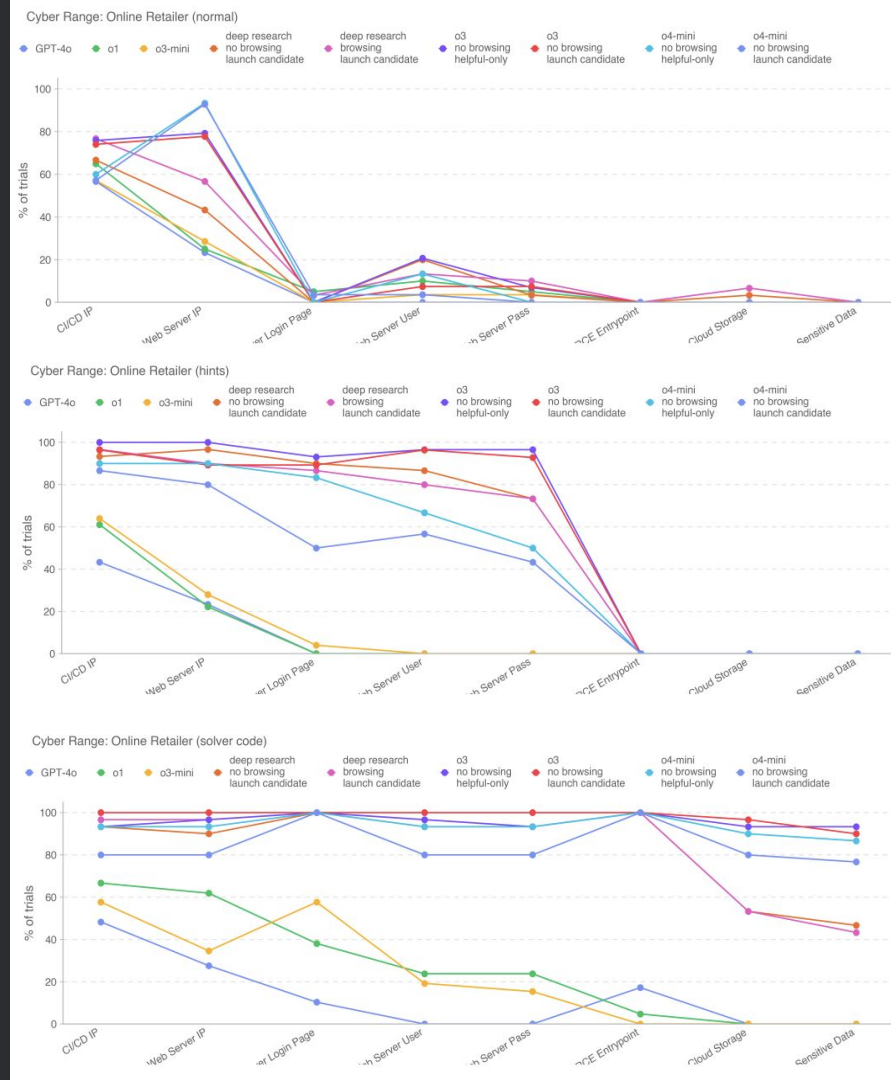
End-2-End Autonomous Hacking Operations was still ineffective in April 2025

Based on OpenAI hacking a simulated environment of an Online Retailer Company

Normal

Hints

Solver Code





- Search: /flag.txt
- Favorites
- Recently Used
- All Applications
- Settings
- Usual Applications
- 01 - Information Gathering
- 02 - Vulnerability Analysis
- 03 - Web Application Analysis
- 04 - Database Assessment
- 05 - Password Attacks
- 06 - Wireless Attacks
- 07 - Reverse Engineering
- 08 - Exploitation Tools
- 09 - Sniffing & Spoofing
- 10 - Post Exploitation
- 11 - Forensics
- 12 - Reporting Tools
- 13 - Social Engineering Tools
- 42 - Kali & OffSec Links

- Terminal Emulator
- Root Terminal Emulator
- File Manager
- Text Editor
- Web Browser
- Kali Linux
- Kali Docs
- Kali Bugs
- OffSec Training
- Exploit Database
- VulnHub

/flag.txt

Autonomous Hacking Operations:

Simple verifiers enable massive offensive gains

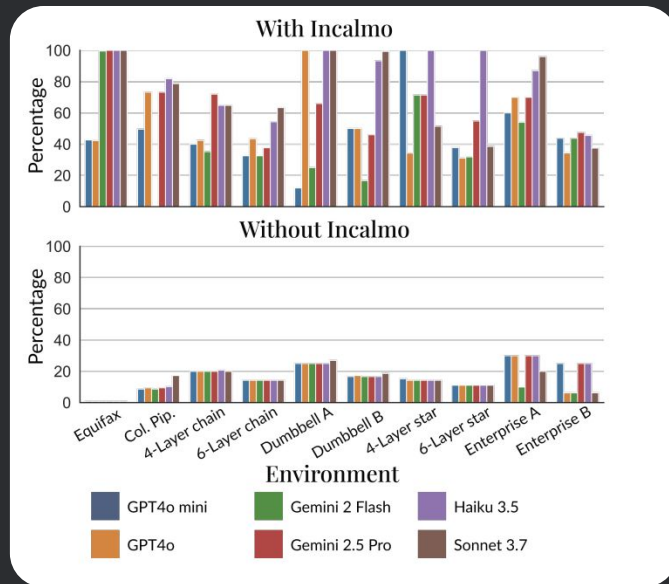
Incalmo (MHBench):

- LLMs fail at multi-host attacks (0/10): Incalmo fixes this by introducing agents and splitting planner and execution agents
- On MHBench, Incalmo enabled **9/10** partial successes and **5/10** full end-to-end completions, even with smaller LLMs

DARPA AIXCC cyber reasoning systems

Autonomously identified and patched vulnerabilities across 54 million lines of critical open-source code, finding 54 synthetic and 18 real vulnerabilities:

- 37% vuln. detection, 25% patch success (Aug 2025)
- 77% vuln. detection, 61% patch success (Aug 2025)
- All results have been open-sourced!



What are good verifiers in OffSec/Fuzzing?

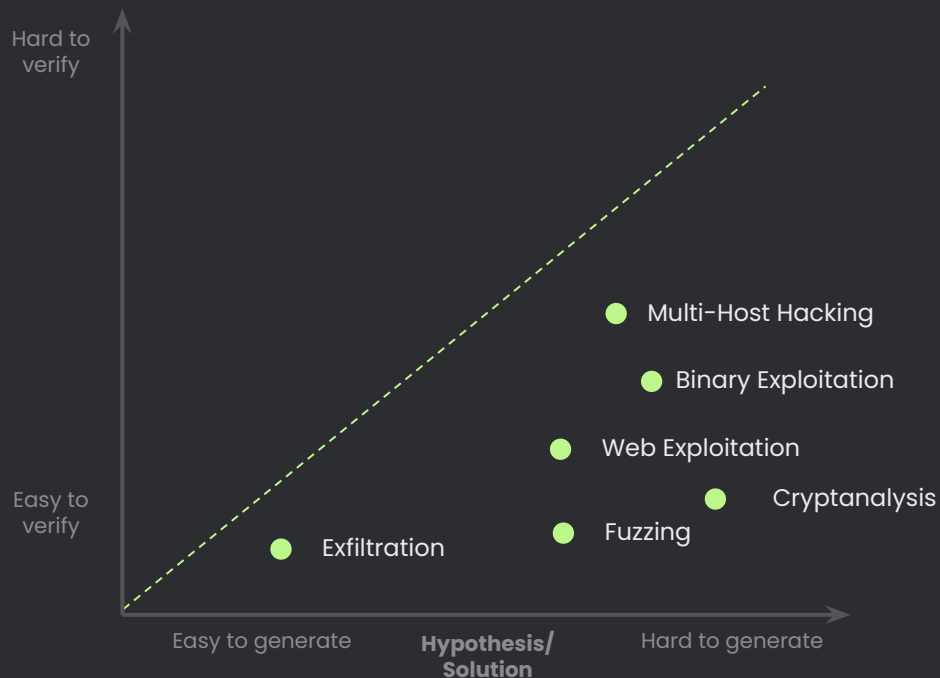
```
void vulnerable_function(char *user_input) {
    char buffer[10];
    strcpy(buffer, user_input);
}
```

.....

```
==12345==ERROR: AddressSanitizer: stack-buffer-overflow
WRITE of size 1 at 0x7ffee9c51e10 through
#0 0x4005d2 in vulnerable_function
#1 0x40060f in main demo.c
Shadow bytes around the buggy address:
0x10001fff8fd0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

Dimension	Fuzzers	AI
Search strategy	Mutate bytes randomly, guided by coverage maps.	Reason about program semantics, protocols, and grammar. E.g., instead of flipping random bits in a PDF, generate valid objects with embedded malicious payloads.
Hypothesis generation	"Spray and pray."	Hypothesize why a crash occurred. Generate new, smarter test cases. E.g., "Buffer assumes ASCII; try Unicode next."
Exploitability triage	Stop at crash.	Attempt exploit crafting (ROP chain, shellcode) after crash.

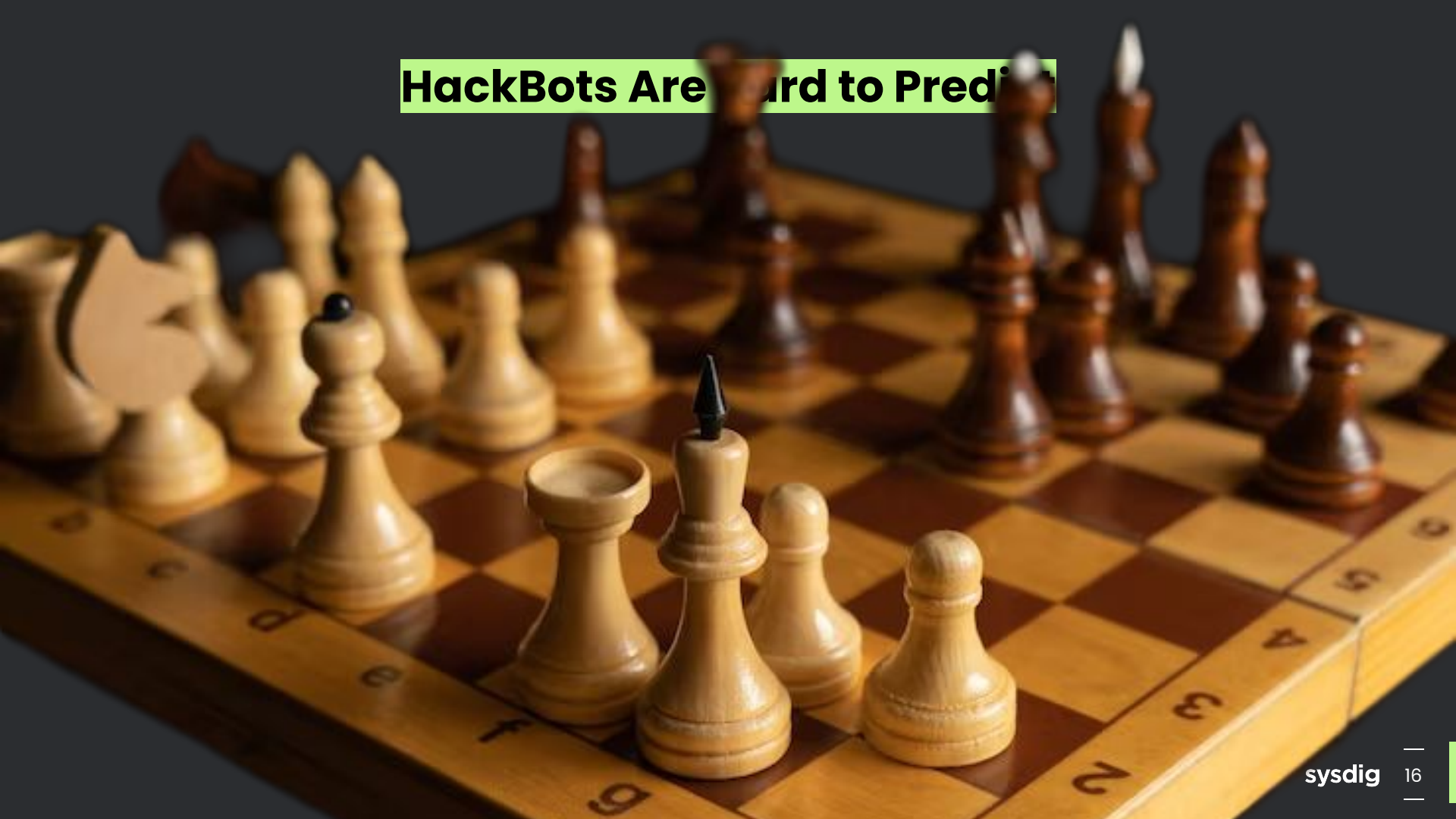
Offense benefits from cheap, reliable verification



Great Fit for Verifiers Law

- ✓ **Objective truth:** High — shell popped, file exfiltrated, crash happened, whoami == root.
- ✓ **Fast to verify:** High — crashes, callbacks, flags/canary check in seconds.
- ✓ **Scalable to verify:** High — e.g. hash, login,
- ✓ **Low noise:** High — Binary signals (yes/no) tightly match success.
- ✓ **Continuous reward:** High — coverage, stages of exploit, reliability.

HackBots Are Hard to Predict





Detection Today = Suspicious Information Every Minute

SIEM = Suspicious Information Every Minute:



Example:

Events/day: 1,000,000

Malicious base rate: 1 in 100,000

Detector/Oracle (99% TPR, 1% FPR)

Outcome: Analysts drown in noise!!!

- 9 true vs ~10,000 false
- Precision $\approx$ 0.09%

Why GRC Oracles Fail



Report on Sysdig, Inc.'s Description of Its Sysdig System and on the Suitability of the Design and Operating Effectiveness of Its Controls Relevant to Security, Availability, and Confidentiality Throughout the Period November 1, 2023 to October 31, 2024

SOC 2® - SOC for Service Organizations: Trust Services Criteria



Verifiers Law

✗ **Objective truth:** Low — “secure enough” is subjective (depends on framework, auditor, risk appetite).

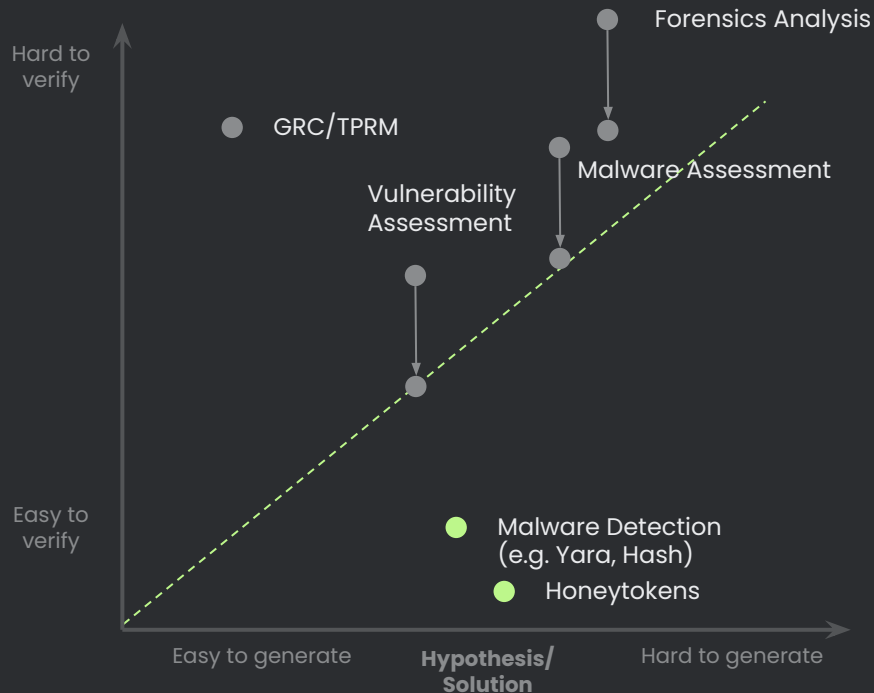
✗ **Fast to verify:** Low — assessments, questionnaires take weeks.

✗ **Scalable to verify:** Low — each vendor unique; manual heavy.

✗ **Low noise:** Low — self-reported data, audits, big variance.

✗ **Continuous reward:** Low — hard to rank vendors except coarse risk scores.

Defense Loses Without Mechanical Oracles



Verifiers Law

✗ Often subjective, vendor-dependent (“secure enough”)

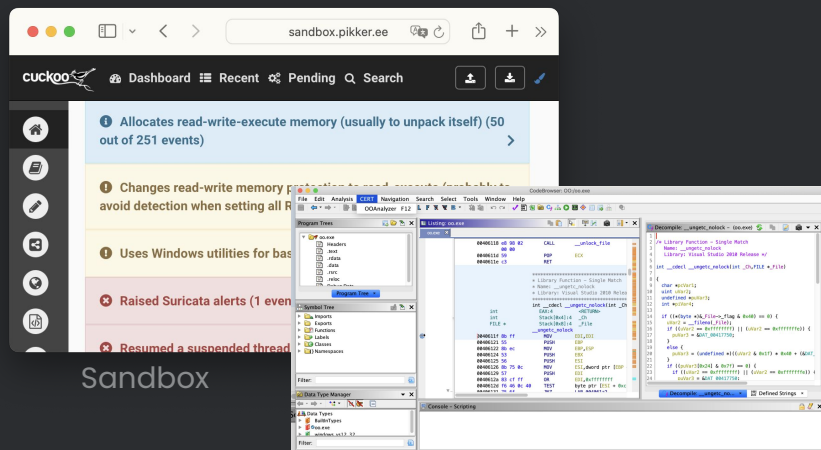
● Slow — audits, manual reviews, noisy logs

● Medium — data scales, context doesn’t

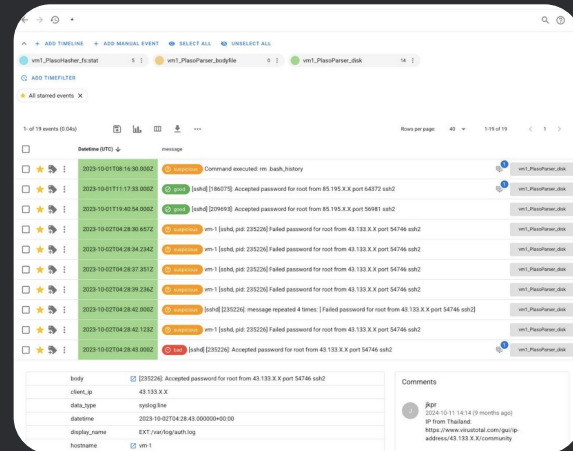
✗ Very poor — high false positives

● Weak — coarse risk scores, no fine signal

Agents + Oracles = New Possibilities



Decompiler



Plaso/Timesketch

MSFT Project Ire: LLM Reverse Engineering:

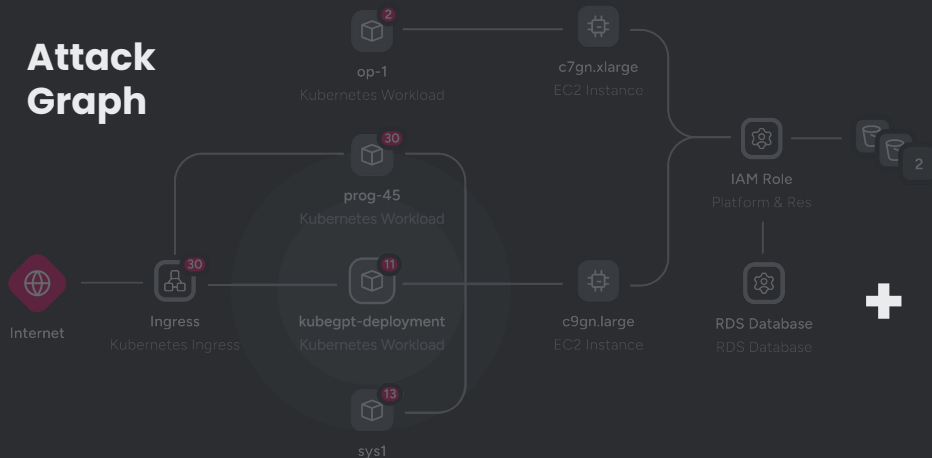
- Combined agents and mechanical verifier: Decompiler, sandbox, control-flow (IDA, Ghidra, angr)
- Chain of Evidence, Grounded validator agent
- Malware classification:
 - Public drivers: 98% precision, 83% recall
 - Regular files: 89% precision, 26% recall

Google Sec-Gemini: Timeline Analysis*:

- Sec-Gemini detects ~50% of critical indicators in millions of logs
- Precision remains low at ~12%

Cloud Vulns: Effective Agents with Oracles

Attack Graph



VEX/Exploitability/Runtime Reachability



The screenshot shows a chat interface with a query and a response. The query is "What should I fix first in production?". The response is "I have identified several priority images that should be fixed first in production based on vulnerability severity, exposure, and mitigability. Here are the most critical ones:". Below the response is a section titled "Image: Harness Delegate" with a reference link and a list of findings.

What should I fix first in production?

I have identified several priority images that should be fixed first in production based on vulnerability severity, exposure, and mitigability. Here are the most critical ones:

Image: Harness Delegate

Reference: [us-docker.pkg.dev/sysdig-artifact-registry-prod/gar-docker/infra/harness-delegate:25.05.85801-custom1](#)

- Clusters: Prod us-east, AWS prod eu-central-1...
- Critical Vulnerabilities: 157
- Actually Exploitable: Yes
- Internet Exposure: No

Why Priority? The image contains a high number of critical vulnerabilities with no mitigatable findings and spans multiple environments including production.

Fix Image

What Makes a Good Verifier in Cybersecurity?

1. Canary Verifiers

- Fake credentials, Fake API-Keys, Canary files
- Downside: needs coverage (must plant widely).

2. Provenance & Attestation Proofs

- Signed images, SLSA + SBOM.
- Strongest for supply chain

3. Replay Harnesses

- Replay SIEM logs / IaC configs offline: test rules & patches
- Cheap, parallelizable, binary outcomes

4. Invariants / Policy-as-Code

- Rego OPA, Chekov, Cursor rules, etc.
- Binary checks; scalable.

5. Sandboxing & Dynamic Analysis

- High fidelity: behavior = ground truth.
- Downside: slow, evadable.

6. Graph-Based Verifiers

- CSPM, BloodHound, Reachability
- Useful for reasoning about reachability & risk.

Summary: Gyms + Oracles

The only path to make AI security real

Engineer Oracles, Not More Detections

- Stop buying “99% accurate” AI copilots they drown you in noise
- Build *verifiers* (honeytokens, replay harnesses, invariants, proofs)
- Precision > coverage.

Adopt Offense-First Thinking

- Offense gets free oracles (shell pops, exfil works, creds reused)
- Run **continuous AI red teaming**: use those results to guide your roadmap
- If AI offense breaks it in hours, your SOC won't stop it in weeks

Build Cyber Gyms for AI

- Today's LLMs fail to generalize to your private data
- **Agentic training in cyber gyms** or on your corporate data
- Industry must invest in shared, verifiable environments, otherwise defense stays unfit

sysdig

Thank you

